TỔNG HỢP YÊU CẦU CÔNG NGHỆ THEO ĐỀ TÀI ĐÃ ĐĂNG KÝ

Bản hướng dẫn chi tiết dành cho sinh viên · Môn Các Công nghệ Lập trình Hiện đại · 09/2026

Trích xuất riêng cho đề tài: Next.js 16 (Track Web Frontend)

TRỌNG TÂM CỦA MÔN HỌC

Công nghệ là đối tượng học và đối tượng được đánh giá. Ứng dụng/đề tài nghiệp vụ chỉ là bối cảnh để nhóm có chỗ áp dụng công nghệ. Đề tài tốt là đề tài có nghiệp vụ **ĐƠN GIẢN, QUEN THUỘC** (đã biết rõ luồng, không mất thời gian đi tìm hiểu nghiệp vụ) để dành toàn bộ công sức cho việc tìm hiểu sâu và làm chủ công nghệ. Nghiệp vụ phức tạp không đem lại điểm cộng; nếu dành nhiều thời gian học nghiệp vụ hơn học công nghệ thì đề tài đang lệch trọng tâm.

PHẠM VI VÀ CÁCH SỬ DỤNG TÀI LIỆU

• Tài liệu này tổng hợp yêu cầu cho công nghệ **Next.js 16 (Track Web Frontend)** được trích xuất từ tài liệu chung của môn học, giúp nhóm tập trung đúng vào trọng tâm kỹ thuật cần đạt.

• Đề tài được đánh giá theo 3 tầng:

- **Tầng 1 (Lõi công nghệ):** chia thành 1A (bắt buộc) và 1B (tự chọn theo mục tiêu ứng dụng). Bắt buộc phải hoàn thành 1A và các mục 1B đã chọn để được đánh giá đạt.
- **Tầng 2 (Chuẩn kỹ nghệ phần mềm dùng chung):** áp dụng cho mọi đề tài theo đặc thù nền tảng, là điều kiện tiên quyết để đạt điểm khá trở lên.
- **Tầng 3 (Nâng cao và đào sâu):** dành cho nhóm muốn đạt điểm giỏi/xuất sắc, yêu cầu có số liệu, đo lường hoặc minh chứng kỹ thuật tương xứng.

QUY TẮC ĐỌC NHANH

Tầng 1A trả lời: “Nếu bỏ những mục này thì còn có thể nói là em đã học công nghệ này không?”
Tầng 1B trả lời: “Ứng dụng của em cần thêm cơ chế nào của chính công nghệ?”
Tầng 2 trả lời: “Em có làm sản phẩm kỹ thuật một cách nghiêm túc không?”
Tầng 3 trả lời: “Em đã đào sâu công nghệ đến đâu, có đo lường/so sánh/phân tích gì vượt mức vận hành cơ bản?”

0. TẦNG 2 — CHUẨN KỸ NGHỆ PHẦN MỀM DÙNG CHUNG

Áp dụng theo đặc thù nền tảng. Không bắt một nền tảng no-code phải Docker hóa hay một ứng dụng managed-auth phải tự hash mật khẩu. Nhóm phải đạt yêu cầu tương đương phù hợp với công nghệ mình chọn.

- **Quản lý phiên bản:** Dùng Git cho dự án có mã nguồn: nhánh làm việc, pull request/merge request, review chéo, lịch sử commit có ý nghĩa; không để toàn bộ nhóm cùng commit thẳng vào main.
- **Khả năng chạy lại:** Có README rõ ràng, phiên bản runtime/package, lệnh cài và lệnh chạy. Tái tạo môi trường chạy ổn định, tường minh.
- **Quản lý cấu hình và bí mật:** API key, mật khẩu, connection string phải qua biến môi trường/secret store; cung cấp `.env.example` nhưng không commit secret thật.
- **Kiểm thử:** Có test phù hợp với sản phẩm: unit test cho logic; component/flow test cho frontend; đảm bảo tính toàn vẹn của mã nguồn.
- **CI và kiểm tra tự động:** Dự án code nên có GitHub Actions hoặc tương đương để build/lint/test khi đẩy mã lên kho chứa.
- **Bảo mật cơ bản:** Biết rủi ro liên quan trực tiếp tới đồ án: auth/authorization, input validation, CORS, injection, XSS, upload file, API key, quyền truy cập dữ liệu. Không cần “học thuộc OWASP” nếu không gắn với hệ thống.
- **Xử lý lỗi và quan sát:** Không để lỗi rơi tự do. Có thông báo lỗi phù hợp; log đủ để truy được luồng chính.
- **Tài liệu kỹ thuật:** README phải giúp một nhóm khác chạy lại được. Mô tả các biến môi trường, API sử dụng, luồng cài đặt và khởi chạy.
- **Sử dụng AI hỗ trợ code:** Được phép dùng AI trong quá trình làm. Nhóm phải kiểm chứng code/claim do AI gợi ý và khai báo theo quy định môn học; không được dùng “AI đã viết” để thay cho khả năng giải thích.

1. TRACK WEB FRONTEND

Định hướng track

Frontend không được đánh giá bằng số lượng trang hay mức độ "đẹp". Nhóm phải chứng minh hiểu mô hình render, state/data flow, routing và cơ chế đặc thù của framework. Tầng 3 dành cho tương tác, hiệu năng, realtime, offline, đồ họa hoặc khả năng truy cập.

Next.js 16

Công nghệ chính: Next.js App Router. React là nền tảng bên dưới; yêu cầu phải thể hiện được những gì làm Next.js khác một React SPA thuần.

Tầng 1A — Lõi bắt buộc

- **App Router:** tổ chức route theo thư mục; hiểu page/layout và cách cây route được dựng.
- **Phân biệt Server Component và Client Component:** biết component nào mặc định chạy phía server, khi nào cần `"use client"`, và tác động của ranh giới này tới dữ liệu/bundle.
- **Lấy dữ liệu phía server trong Server Component:** không biến mọi trang thành client component chỉ để gọi fetch.
- **Hiểu ít nhất luồng render chính của chính đồ án:** request vào route nào, dữ liệu lấy ở đâu, phần nào render server/phần nào cần tương tác client.

Tầng 1B — Lõi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- Route động/lồng nhau; layout dùng chung; loading/error/not-found phù hợp với luồng ứng dụng.
- Caching và revalidation: chọn chiến lược phù hợp dữ liệu; biết khi nào cần dữ liệu tươi và khi nào có thể cache.
- Server Action/Server Function cho mutation hoặc Route Handler khi cần API endpoint trong cùng dự án.
- Suspense/streaming để chia nhỏ thời điểm hiển thị dữ liệu chậm.
- Tối ưu Image/Font; metadata/SEO nếu mục tiêu sản phẩm cần.
- Middleware cho redirect/rewrite/check đơn giản; không lạm dụng middleware để chứa toàn bộ business logic.

Minh chứng tối thiểu cần có

- Chỉ ra ít nhất một route thật trong đồ án và giải thích server/client boundary của route đó.
- Có code và ảnh/log chứng minh data fetching hoạt động ở đúng phía đã thiết kế.
- Nếu dùng caching/revalidation hoặc Server Action/Route Handler, phải có tình huống chạy chứng minh trước/sau thay đổi.

Chưa được xem là đạt nếu chỉ...

- Chỉ tạo nhiều trang React rồi đặt vào thư mục app nhưng toàn bộ đều là Client Component.
- Dùng Next.js chỉ như router, trong khi dữ liệu vẫn được fetch hoàn toàn kiểu SPA và không giải thích được vì sao.
- Kể tên SSR/SSG/ISR nhưng đồ án không thể chỉ ra cơ chế render/caching nào đang thật sự dùng.

Câu hỏi tự kiểm

- "Cái gì trong đồ án này là Next.js mà một React SPA bình thường không làm giống như vậy?"
- "Nếu bỏ `"use client"` ở component này thì chuyện gì xảy ra? Tại sao?"
- "Dữ liệu này nên cache hay luôn lấy mới? Quyết định đó ảnh hưởng gì?"

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đồ án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thêm một dynamic route vào module có sẵn và xử lý not-found.
- Chuyển một thao tác cập nhật từ client fetch sang Server Action/Route Handler phù hợp với code hiện tại.
- Giải thích và chỉnh lại một server/client boundary chưa hợp lý trong chính đồ án.

TẦNG 3 — NÂNG CAO CHUNG CHO WEB FRONTEND

Chỉ chọn khi phù hợp. Tầng 3 không phải danh sách "phải đủ 2 mục"; độ sâu và minh chứng quan trọng hơn số lượng.

- **Kéo thả:** Xử lý drag/drop có state thật, lưu được thứ tự hoặc cấu trúc sau thao tác; không chỉ demo kéo một ô màu.
- **Hoạt hoạ:** CSS View Transitions/Motion/GSAP… dùng có mục đích; nếu nói tối ưu phải quan sát bundle hoặc trải nghiệm chứ không chỉ thêm hiệu ứng.
- **3D/Canvas/Data visualization:** Three.js/R3F/Konva/D3; cần có tương tác hoặc dữ liệu thật của demo, đồng thời kiểm soát hiệu năng.
- **Danh sách lớn/virtualization:** Dùng virtual list cho dữ liệu đủ lớn để thấy khác biệt; có so sánh render/scroll trước–sau.
- **Realtime/cộng tác:** WebSocket/SSE/CRDT; cần xử lý kết nối, cập nhật trạng thái, reconnect hoặc conflict ở mức phù hợp.
- **Rich editor:** TipTap/Lexical/Slate; cần dữ liệu có cấu trúc, lưu/khôi phục nội dung, không chỉ dùng editor mặc định.
- **PWA/offline:** Service Worker/IndexedDB; phải chứng minh ứng dụng vẫn xử lý được một luồng khi mất mạng và đồng bộ lại khi có mạng.
- **Accessibility/performance:** Kiểm tra keyboard/focus/ARIA hoặc Lighthouse/Core Web Vitals theo phạm vi; có số liệu và sửa ít nhất một vấn đề cụ thể.

5. CHECKLIST NHÓM PHẢI HOÀN THÀNH TRƯỚC KHI NỘP

1. Ghi đúng tên công nghệ chính trong tên đề tài và báo cáo; không chỉ ghi tên bài toán nghiệp vụ.
2. Đánh dấu các mục Tầng 1A đã hoàn thành và chỉ rõ file/module/route/screen nơi thể hiện.
3. Liệt kê các mục Tầng 1B nhóm chủ động chọn vì phù hợp ứng dụng; nêu lý do chọn.
4. Liệt kê các yêu cầu Tầng 2 đã thực hiện và minh chứng chạy/test/repo.
5. Liệt kê Tầng 3 nếu có; mỗi mục phải kèm số liệu, test case hoặc minh chứng kỹ thuật tương xứng.
6. Bảng phủ kỹ thuật trong báo cáo phải khớp code đã nộp; không khai kỹ thuật chỉ có trong slide hoặc demo rời.
7. Chuẩn bị để bất kỳ thành viên nào cũng có thể tìm đúng code, sửa một yêu cầu nhỏ và giải thích cơ chế mà không dùng AI.
8. Kiểm tra lại phạm vi nghiệp vụ: nếu nhóm vẫn dành phần lớn thời gian cho quy trình/dữ liệu nghiệp vụ thay vì công nghệ, hãy đơn giản hóa trước khi quá muộn.

> **NHỚ LẠI MỘT CÂU**
>
> *Bài toán chỉ cần đủ đơn giản để công nghệ có chỗ thể hiện. Điểm cao đến từ việc hiểu đúng, dùng đúng và đào sâu công nghệ — không đến từ việc xây một "hệ thống nghiệp vụ thật" thật lớn.*